

COMMON PROBLEMS IN A VIBE-CODED CODEBASE

Practical bug, architecture, security and production-readiness audit guide

Purpose

Use this document to audit a codebase built heavily with AI coding agents, rapid prompting or “vibe coding”. The goal is not to assume AI-generated code is bad; it is to systematically look for the recurring failure modes that become easy to accumulate when implementation moves faster than architecture, testing and review.

Important: Many of these are ordinary software-engineering problems that existed before AI. Vibe coding mainly changes the speed, scale and probability with which they can accumulate when the developer does not deliberately verify the result.

Includes synthesized learnings from the original three r/vibecoding discussions plus a wider subreddit review of architecture, security, database, testing, deployment, maintainability, observability and AI-agent workflow threads. Reddit observations are anecdotal community evidence and are used as practical audit signals, not statistics.

Architecture & Structure

Look for structural choices that make every later feature harder, riskier or more duplicated.

Common problem	Small explanation / what to look for
Duplicate functionality	The same feature or helper is implemented more than once instead of reusing a canonical implementation.
Multiple sources of truth	The same state, setting or business rule exists in several places and eventually becomes inconsistent.
God classes / giant files	One class, component or file gradually takes responsibility for too many unrelated behaviours.
Poor separation of concerns	UI, persistence, validation, calculations and business rules are mixed together.
Inconsistent architecture	Different features use different patterns because they were generated independently.
Feature-specific hacks	A one-off conditional fixes a symptom instead of correcting the underlying design.
Circular dependencies	Modules depend on each other in both directions, making changes risky and testing difficult.
Over-coupled components	Changing one component unexpectedly breaks unrelated parts of the application.
Unnecessary abstractions	Interfaces, factories, wrappers or service layers are created without a real need.
Missing abstractions	Repeated behaviour that should be centralized remains duplicated across the project.
Parallel implementations	A new implementation is added beside the old one instead of modifying or replacing it.
Architecture drift	New AI-generated code stops following the project's original design rules and conventions.

Code Duplication

Duplication is especially dangerous in agent-built code because later prompts may update only one copy.

Common problem	Small explanation / what to look for
Copy-pasted methods	Nearly identical methods appear across multiple files and gradually diverge.
Duplicate utility functions	Several helpers perform the same conversion, calculation or formatting task.
Duplicate UI components	Equivalent controls are independently implemented and behave slightly differently.
Duplicate validation rules	Different screens or endpoints validate the same data using different rules.
Duplicate constants	Limits, paths, names, IDs and dimensions are repeated instead of centralized.
Duplicate models	Several classes represent the same concept with slightly different fields.
Duplicate queries	The same database or API query logic is rebuilt in multiple places.
Duplicate transformations	Mapping or serialization logic is copied instead of using one canonical mapper.

State Management

State defects create “random” behaviour because different parts of the application disagree about what is true.

Common problem	Small explanation / what to look for
State stored in too many places	UI state, model state and application state contain copies of the same value.
Stale state	The screen shows an old value because a change was not propagated everywhere.
Hidden global state	Static or global variables influence behaviour from locations that are hard to trace.
Uncontrolled mutation	Objects can be changed from many unrelated parts of the program.
Initialization-order bugs	Code uses state or services before they are fully initialized.
State not reset	Opening another file, screen or project leaves values from the previous session behind.
State recreated unnecessarily	Refreshes rebuild state and lose selection, filters, scroll position or edits.
Derived state stored separately	A calculated value is stored rather than derived, so it can disagree with its inputs.

UI & Interaction Bugs

Fast UI generation often produces convincing screens before event lifecycle, resizing and interaction behaviour are robust.

Common problem	Small explanation / what to look for
UI and model out of sync	The interface displays one value while the underlying data contains another.
Incorrect event handling	Clicks, keys, drag/drop or selection events fire unexpectedly or in the wrong order.
Duplicate event subscriptions	The same handler is attached more than once, causing one action to execute repeatedly.
Event handlers never removed	Old windows or objects remain referenced, producing memory leaks and ghost behaviour.
UI rebuilt instead of updated	Entire views or grids are recreated when only a small change was needed.
Hard-coded positioning	Controls break when the window, DPI or font size changes.
Broken resizing	Rows, columns and controls do not correctly react to the available space.
Focus bugs	Keyboard input goes to the wrong control or focus disappears after an update.
Selection bugs	Refreshing or reordering unexpectedly changes the selected item.
Scroll position resets	A normal refresh jumps the user back to the top or another position.
Drag/drop index errors	Reordering by list index moves the wrong item after previous changes.
Coordinate-system confusion	Screen, window and control coordinates are mixed, causing incorrect placement.
DPI scaling bugs	The UI works at 100% scaling but overlaps or clips at other Windows scaling levels.

Data Binding

For MVVM/WPF-style applications, silent binding failures can make a correct model appear broken or vice versa.

Common problem	Small explanation / what to look for
Missing property notifications	The model changes but the UI never receives a notification to refresh.
Incorrect binding paths	Bindings silently point to missing or renamed properties.
Wrong DataContext	A control receives a different object than the binding expects.
Wrong binding direction	Two-way binding lets the UI overwrite values that should be read-only.
Non-notifying collections	A list changes internally but the UI is not told that items were added or removed.
Code-behind fights bindings	Bindings and imperative UI code both try to control the same property.
Converters hide logic	Important business rules are buried inside UI converters.
Binding errors ignored	Runtime binding warnings accumulate without being treated as defects.

Logic & Calculation Bugs

These are small implementation mistakes that can produce convincing but incorrect results.

Common problem	Small explanation / what to look for
Off-by-one errors	Indexes, loops, pages or date ranges are one position out.
Boundary handling failures	Behaviour works normally but fails at minimum, maximum, empty or first/last values.
Incorrect boolean logic	AND, OR and negation combinations produce unintended cases.
Missing null handling	Code assumes an object or value always exists.
Excessive null fallbacks	Repeated null checks hide the real reason data was not initialized correctly.
Silent default values	Failures return zero, empty text or false and make incorrect results look legitimate.
Magic numbers	Unexplained numbers are embedded directly in formulas or conditions.
Unit mismatches	Seconds/minutes, bytes/MB, percentages/decimals or pixels/DPI are mixed.
Integer division	Calculations truncate unexpectedly because integer math is used.
Floating-point equality	Exact equality is used where a tolerance is required.
Incorrect ordering logic	Sorting or reordering fails after repeated moves or equal values.

Async, Threading & Concurrency

Concurrency problems are hard to reproduce and are commonly hidden by simple manual testing.

Common problem	Small explanation / what to look for
Missing await	Work starts asynchronously but later code runs before the operation finishes.
async void misuse	Exceptions become difficult to catch and callers cannot await completion.
Blocking async calls	.Result or .Wait() can freeze the UI or deadlock the application.
UI-thread violations	Background work tries to update UI objects directly.
Race conditions	Two operations read or update the same state at the same time.
Overlapping requests	Repeated clicks or refreshes start multiple competing operations.
No cancellation	Old searches or calculations continue even after they are no longer relevant.
Incorrect cancellation handling	Normal cancellation is surfaced as an application error.
Fire-and-forget tasks	Background tasks fail without a caller observing the exception.
Unprotected shared collections	Multiple threads modify a collection that was not designed for concurrent access.

Error Handling & Observability

A production bug you cannot see, reproduce or trace is much more expensive than a bug that fails loudly with context.

Reddit signal: the specific tool is less important than making sure production errors land somewhere a human actually reviews, with enough information to trace the cause.

Common problem	Small explanation / what to look for
Empty catch blocks	Exceptions are swallowed completely, hiding real defects.
Catch-all exception handling	Specific failures are collapsed into generic handling that loses useful context.
Logging without recovery	An error is written to a log but the application remains in an invalid state.
Generic user errors	Messages such as 'Something went wrong' make support and diagnosis difficult.
Missing context in logs	The log omits the operation, record, user action or inputs that caused the failure.
Exceptions as control flow	Expected situations are handled by throwing and catching exceptions.
Errors converted to defaults	Bad data is replaced with empty or zero values instead of stopping or reporting.
No production error tracking	Crashes and runtime failures are invisible unless a user reports them.
No tracing across operations	It is difficult to connect a production symptom to the request, job or generated commit that caused it.

Files, Persistence & Databases

Persistence defects are high-impact because they can corrupt, lose or silently misread user data.

Common problem	Small explanation / what to look for
Non-atomic saving	A crash during save can leave the entire project file partially written or corrupted.
No pre-save backup	A bad write can permanently replace the last known-good version.
Hard-coded paths	The application only works on the original developer's machine.
Relative-path confusion	File access depends on the current working directory.
Undisposed file handles	Files remain locked because streams are not closed reliably.
Undisposed DB connections	Connections accumulate until operations slow down or fail.
N+1 queries	A screen performs hundreds of queries where a small number would be sufficient.
Load-everything queries	Entire tables are loaded into memory because the test dataset was small.
Schema compatibility failures	Older saved data stops loading after models or schemas change.
Unsafe serialization assumptions	Renaming or removing a property unexpectedly breaks saved files.
Partial transactions	Only part of a multi-step change is saved when one operation fails.

Performance & Scaling

Small demo datasets often hide design choices that fail abruptly when real data volume arrives.

Reddit signal: one cleanup thread specifically called out loading an entire table because the agent had only seen ~20 test rows; the screen later stalled with thousands of records.

Common problem	Small explanation / what to look for
Repeated recalculation	Expensive calculations rerun whenever any nearby value changes.
Rendering too often	Charts, grids or complex views redraw many times for one logical update.
No batch updates	Thousands of individual notifications are sent rather than one grouped change.
Allocation-heavy hot paths	Temporary lists, objects or strings are continuously recreated.
Repeated disk reads	The same file is loaded again and again instead of being cached appropriately.
Repeated identical queries	The application refetches unchanged data unnecessarily.
No UI virtualization	Thousands of off-screen controls are created even though the user cannot see them.
Memory leaks	Timers, events, tasks or window references keep dead objects alive.
No pagination	A feature that works with 20 test rows becomes unusable with thousands of real records.
Unbounded collections	Logs, caches or history lists grow forever.

Configuration & Environment

Environment assumptions make code work on the creator's machine and fail in deployment or on another user's PC.

Common problem	Small explanation / what to look for
Hard-coded configuration	URLs, limits, paths, feature flags or environment settings are compiled into the code.
Duplicated configuration	Different files contain competing values for the same setting.
Missing defaults	The application crashes when an optional configuration field is absent.
No config migration	Updates break settings created by earlier versions.
Debug settings in production	Test endpoints, verbose logging or developer switches remain enabled.
Environment assumptions	Code assumes a particular OS, locale, folder structure, port or installed tool.
Mixed development/production values	Production code accidentally uses local endpoints, test accounts or fake data.

Naming, Readability & Maintainability

Readable code reduces dependence on the original prompt history and helps both humans and AI reason correctly later.

Common problem	Small explanation / what to look for
Meaningless names	Variables such as data, result2 or temp hide what the value represents.
Inconsistent terminology	The same concept is called Task, Item, Activity and Job in different modules.
Very long methods	Generated methods keep absorbing responsibilities and become hard to reason about.
Misleading method names	A function named LoadData also saves, updates UI and triggers calculations.
Outdated comments	Comments describe behaviour that no longer exists.
Obvious AI comments	Large amounts of narration add noise but not useful intent or constraints.
Hidden side effects	Methods mutate unrelated state without their name or signature making that clear.
Inconsistent coding conventions	Generated code switches styles, patterns and error-handling conventions between files.

API & Service Layer

Treat the service boundary as a contract: validate inputs, define failures and isolate external dependencies.

Common problem	Small explanation / what to look for
Business logic inside API clients	Networking code starts deciding application behaviour and domain rules.
No timeout	A request can wait indefinitely and make the application appear frozen.
Bad retry policy	Temporary failures either fail immediately or retry aggressively without limits.
Response-shape assumptions	Code assumes every API response contains every field and the expected type.
API models mixed with domain models	External service changes ripple through the entire application.
Missing idempotency	Retrying a request can accidentally create duplicate records or charges.
Unvalidated inputs	Endpoints accept data without checking format, range or business constraints.
Inconsistent error contracts	Different endpoints represent failures in incompatible ways.

Security & Authorization

The server must enforce security independently of what the UI shows or allows.

Reddit signal: repeated examples focused on “UI-only security” and users being able to access another user’s record by changing an ID. Treat these as critical audit tests.

Common problem	Small explanation / what to look for
Secrets committed to source	API keys, passwords or tokens appear in code, configuration or Git history.
Secrets exposed in frontend code	A browser or client application receives values that should only exist on the server.
UI-only security	Admin buttons are hidden, but the backend endpoint itself has no authorization check.
Missing resource ownership checks	An authenticated user can request another user’s record simply by changing an ID.
SQL injection	Untrusted values are concatenated directly into database queries.
Command injection	Untrusted input is inserted into shell or system commands.
Path traversal	User-controlled paths can escape the intended storage directory.
Unsafe deserialization	Untrusted data is converted into complex objects without safe constraints.
Sensitive information in logs	Tokens, passwords or private data are written to logs.
Client-side-only validation	Security depends on what the UI allows instead of what the server enforces.
Security comments without code	Generated comments promise an ownership or permission check that the implementation never performs.

Authentication & Session Management

Authentication is a poor area to improvise. Prefer mature frameworks and explicit lifecycle rules.

Reddit signal: commenters highlighted home-grown JWT/session/reset logic as a recurring weak point. Mature authentication components reduce the amount of security behaviour you must invent and verify yourself.

Common problem	Small explanation / what to look for
Reinvented authentication	Custom authentication is written instead of using a mature, well-tested framework or provider.
JWTs with unsafe lifetime	Tokens are issued without sensible expiry, rotation or revocation behaviour.
Reset tokens that never expire	Password-reset or magic-link tokens remain valid indefinitely.
Weak session invalidation	Changing a password or role does not terminate existing sessions.
Frontend-only role checks	The UI knows the user's role but server actions do not verify it.
Inconsistent auth middleware	Some endpoints use authorization checks and others accidentally bypass them.
Account enumeration	Login or reset flows reveal whether a particular account exists.
Missing brute-force protection	Authentication endpoints can be repeatedly attempted without throttling or lockout controls.

Payments & Trust Boundaries

Anything supplied by a client should be treated as untrusted until verified server-side.

Reddit signal: do not accept price, entitlement or payment-success claims from the client; verify provider events and signatures on the server.

Common problem	Small explanation / what to look for
Client-controlled prices	The frontend sends the amount to charge and the server trusts it.
Unverified payment webhooks	Payment events are accepted without verifying the provider's signature.
Duplicate payment processing	Retries or repeated webhook delivery can create duplicate actions.
Entitlements based on UI state	Paid access is granted because the client says payment succeeded.
No reconciliation	Application records can drift away from the payment provider's actual state.
Test/live environment confusion	Test keys, live keys, products or webhook endpoints become mixed.

Dependencies & Packages

AI-generated examples can introduce packages quickly; each dependency adds maintenance, security and compatibility cost.

Common problem	Small explanation / what to look for
Duplicate libraries	Several packages solve the same problem and introduce overlapping behaviour.
Heavy package for tiny feature	A large dependency is added to avoid writing a small amount of straightforward code.
Abandoned dependencies	Generated examples rely on obsolete or unmaintained packages.
Version conflicts	Packages require incompatible versions of shared dependencies.
Unused packages	Dependencies remain after the feature that required them is removed.
Framework functionality reimplemented	Custom code duplicates a stable feature already provided by the platform.
Unpinned or uncontrolled versions	A fresh install can pull different behaviour than the version that was tested.

Testing & Regression Protection

Testing converts “it seems to work” into repeatable evidence and prevents old bugs returning after new prompts.

Reddit signal: visual/screenshot checks were recommended to catch “silent rewrites” where the AI changed pages outside the area the developer was watching.

Common problem	Small explanation / what to look for
No automated tests	Every change relies on manually opening the application and hoping important paths still work.
Happy-path-only tests	Normal inputs pass while empty, invalid, concurrent or unusual user behaviour is never tested.
Tests mirror implementation	The test repeats the code's logic and therefore confirms the same mistake.
Tests depend on each other	Running tests individually produces different results from running the full suite.
Flaky tests	Timing, random data or shared state makes results intermittent.
No regression tests	A fixed bug can return because no test captures the expected behaviour.
Mocks everywhere	Tests verify mocked behaviour but never prove that real integrations work.
No user-journey tests	Critical flows such as sign-up, save, checkout or permissions are not tested end to end.
No visual regression tests	AI silently changes important screens and nobody notices until later.
No migration tests	Database or saved-file upgrades are not tested against older real data.

AI / Vibe-Coding Specific Failure Modes

These failures arise from context limits, agent behaviour and rapid prompt-driven iteration rather than any single language or framework.

Reddit signal: users described code quality worsening as projects grew, context became crowded and fixes accumulated on top of earlier fixes. This is a strong reason to stop and consolidate rather than continue patching.

Common problem	Small explanation / what to look for
AI creates instead of modifying	The model adds a second system because it did not understand or locate the existing one.
Context loss	The AI makes a locally sensible change without seeing the code that makes it globally wrong.
Hallucinated APIs	Generated code references methods, packages or framework behaviour that does not exist.
Partial refactors	Only some callers or modules are moved to the new design.
Patch-on-patch development	Each bug adds another conditional or workaround instead of fixing the root cause.
Prompt-induced duplication	Asking for a feature again causes it to be implemented again.
Silent rewrites	The agent changes unrelated areas that were not part of the requested task.
Existing conventions ignored	New code stops following established naming, architecture or error-handling patterns.
Unnecessary rewrites	Working code is replaced because the AI prefers a different implementation.
Feature regressions	A new request silently removes or changes an older requirement.
Fake completeness	Methods look finished but return placeholder, fallback or dummy data.
Commented-out legacy code	Old implementations are left in place instead of relying on version control.
Excessive defensive code	Null checks, broad catches and fallbacks hide architecture defects.
Behaviour changed to pass tests	The generated fix changes expected behaviour rather than fixing the underlying bug.
AI explains rather than verifies	A confident explanation is accepted without tracing the actual execution path or running the relevant checks.

Refactoring & Change Management

Refactors fail when only part of the codebase moves to the new design.

Common problem	Small explanation / what to look for
Incomplete signature changes	Some callers still use the old method or constructor contract.
Incomplete renames	Old names remain in XAML, JSON, SQL, config, tests or serialization.
Dead code after refactor	The replacement exists but the legacy implementation remains in the project.
Feature work mixed with refactors	Large unrelated changes make regressions difficult to isolate.
No compatibility migration	Older projects, files or settings stop working after an internal redesign.
Temporary adapters become permanent	Compatibility wrappers accumulate instead of being removed after migration.
No deprecation plan	Old interfaces remain usable forever and create multiple supported ways to do the same thing.

Date, Time & Locale

Date/time bugs are quiet, location-dependent and often invisible to a team testing in one timezone.

Reddit signal: a cleanup commenter described local-time storage later being interpreted as UTC, quietly breaking date filters for users in other regions.

Common problem	Small explanation / what to look for
UTC/local-time confusion	Stored timestamps and displayed timestamps are interpreted in different time standards.
Timezone ignored	Users in different regions receive incorrect dates, filters or scheduled behaviour.
Daylight-saving bugs	Fixed UTC offsets are used instead of real timezone rules.
Dates compared as text	Sorting and filtering produce incorrect chronological results.
Month-length assumptions	Calculations assume every month has the same number of days.
Device-time dependence	Business rules use the client clock, which may be wrong or in another timezone.
Locale-sensitive parsing	Dates or numbers parse differently on machines using another regional format.

Collections, Identity & Indexing

Stable identity matters whenever rows can be sorted, filtered, reordered, deleted or reloaded.

Common problem	Small explanation / what to look for
Index used as identity	Reordering a list causes references to point to the wrong logical object.
Collection modified during iteration	Items are added or removed while looping, producing exceptions or skipped entries.
Duplicate IDs	Different records accidentally receive the same identifier.
IDs regenerated on load	Relationships break because stable identifiers are recreated each session.
Reference/copy confusion	Editing one object unexpectedly changes another because they share the same reference.
Unstable sort keys	Equal sort values make items jump around between refreshes.
Case-sensitive identity mismatch	Identifiers that should be equivalent are treated differently because casing rules are inconsistent.

User Data Integrity, Backups & Migrations

Protect the user's data first: validate, transact, back up, restore-test and migrate deliberately.

Reddit signal: a backup that has never been restored is unproven. Migration testing was also raised as essential once an application begins evolving.

Common problem	Small explanation / what to look for
Invalid state saved	Incomplete or contradictory objects are persisted instead of being rejected.
Partial operations	A multi-step change applies only some steps before failure.
Undo does not restore full state	Related values remain changed after an undo operation.
Dangling references after delete	Other records still point at an object that no longer exists.
Imports overwrite data	Name or ID collisions are not detected before imported data replaces existing records.
Autosave captures temporary state	Data is persisted halfway through a drag, edit or multi-step operation.
Backups never restore-tested	A backup exists, but nobody has proven it can actually recover the system.
No migration path	Schema or model changes are deployed without transforming existing user data.
Migration not tested on real-size data	An upgrade works on a tiny test database but fails or times out in production.

Version Control & Release Practices

Good history makes AI changes reviewable, reversible and traceable when something breaks later.

Common problem	Small explanation / what to look for
Huge commits	Hundreds of generated changes land together, making review and rollback difficult.
Meaningless commit messages	History consists of messages such as fix, changes or update.
Generated/build files committed	Temporary outputs and machine-specific files pollute the repository.
Formatting mixed with logic changes	Large formatting diffs hide the behaviour that actually changed.
Secrets remain in history	Deleting a key from the latest file does not remove it from prior commits.
No small rollback points	Long AI sessions create one giant change that is difficult to bisect.
No release checklist	Security, migrations, backups, tests and configuration are not systematically checked before deployment.

Production Readiness & Operational Maturity

A working build is not the same as an operable product. Production requires monitoring, recovery and realistic failure handling.

Reddit signal: several comments distinguished building the first version from maintaining and evolving it; real users and unusual behaviour reveal problems that a demo never exercises.

Common problem	Small explanation / what to look for
Works only on the happy path	The demo succeeds, but unusual user behaviour, failures and bad inputs were never designed for.
No deployment verification	A successful build is treated as proof that production actually works.
No monitoring or alerting	The application can fail for hours or days without anyone knowing.
No restore drill	Disaster recovery exists only on paper.
No capacity assumptions	The design has never been tested with realistic user counts or data volumes.
No incident trail	There is no reliable way to connect a production bug to the change that introduced it.
No ownership of operational tasks	Backups, alerts, migrations and dependency updates have no explicit process.
Production treated like development	Debug shortcuts, test data and manual fixes become normal operating procedure.

Observability & Incident Diagnosis

A system that cannot explain what happened to a specific user is difficult to operate safely, and AI will often “fix” symptoms when the evidence is missing.

Reddit-derived learning: Several builders described blind re-prompting when there were no logs or traces. The useful test is whether a real production complaint can be traced quickly to the failing request, job, user action and dependency.

Common problem	Small explanation / what to look for
No structured production logging	Errors and important state changes are printed inconsistently or only to a local console.
No request or correlation IDs	A user action cannot be followed across API calls, jobs, database writes and third-party services.
No user/action context	Logs record an exception but not which account, record or operation triggered it.
No exception tracking	Production failures are not aggregated, deduplicated or surfaced to someone who will act on them.
No operational metrics	Latency, error rate, queue depth, query time and external-service failures are invisible.
No audit trail for critical changes	Important manual/admin actions cannot later be reconstructed reliably.
Browser-console-only diagnosis	Useful errors disappear with the user session instead of reaching centralized monitoring.
AI debugging without evidence	The model is repeatedly asked to guess a fix instead of being given logs, traces, inputs and reproduction steps.

Idempotency, Retries & Duplicate Operations

Real users double-click, refresh, retry and lose connections. Important operations must be safe to repeat or able to detect that they already succeeded.

Reddit-derived learning: A recurring production-vs-demo lesson was that jobs can run twice, webhooks can arrive late or more than once, and users can repeat actions while the first attempt is still completing.

Common problem	Small explanation / what to look for
Non-idempotent writes	Repeating the same request creates duplicate records or applies the same state change twice.
Double-submit bugs	Two fast clicks create two orders, messages, payments or background jobs.
Duplicate webhook processing	The same provider event is accepted more than once with repeated side effects.
Retry-after-timeout duplication	The client retries because it saw a timeout even though the first request actually completed.
No operation/event IDs	The backend has no stable key for determining whether an operation has already been processed.
Partial transaction state	Half an operation succeeds and there is no rollback, resume or reconciliation path.
Overlapping background jobs	Schedulers or workers process the same logical job concurrently without a lock or claim mechanism.
No retry policy	Transient failures either fail permanently or are retried blindly with no backoff, cap or duplicate protection.

Database Design & Data-Model Drift

AI-generated schemas can look clean initially while the application gradually develops duplicated concepts, weak constraints and query patterns the original design never anticipated.

Reddit-derived learning: One recurring subreddit warning was “data-model drift”: duplicated fields, excessive nullability and different screens reading different representations of the same concept.

Common problem	Small explanation / what to look for
Data-model drift	New features add fields/tables without reconciling them with the existing canonical model.
Duplicate semantic fields	Two or more columns represent the same business concept with slightly different names or meanings.
Nullable-by-default design	Fields become optional to make code “work,” allowing invalid or ambiguous states into the database.
Missing constraints	Uniqueness, ownership, foreign-key and state rules exist only in application code or not at all.
Query patterns ignored	The schema was designed before real access patterns were known and is never revisited.
Missing indexes	Queries work on tiny datasets but degrade sharply as row counts grow.
Index-everything reaction	Performance fixes add indexes indiscriminately, increasing write cost and storage without helping real queries.
N+1 query patterns	One logical operation triggers many small database calls rather than a bounded query set.

Database Migrations & Deployment Synchronization

Application code and database schema must change as one controlled release. A tiny schema edit can break every read/write if deployed code expects a migration that has not run.

Reddit-derived learning: A reported outage came from adding one nullable column while the deploy process rebuilt/restarted the app but never ran the database migration.

Common problem	Small explanation / what to look for
Code deployed before schema	The application references a column/table that production does not have yet.
Schema deployed before compatible code	A migration removes or changes something still required by the running application.
Migration ordering assumptions	Later migrations depend on earlier changes that may not have completed everywhere.
Environment schema mismatch	Local, test, staging and production databases are on different migration levels.
Destructive migration generation	AI chooses drop/recreate approaches that risk data loss instead of safe incremental changes.
No pre-deploy schema check	The release process does not detect when a diff changes models/schema and therefore requires migration work.
No rollback/forward-fix plan	A failed migration has no tested recovery strategy.
Migration never tested on realistic data	Changes succeed on an empty/dev database but lock, time out or fail on production-sized data.

Dependency & Supply-Chain Risk

AI agents can add dependencies quickly, but every installed package executes someone else's code and creates ongoing patch, provenance and compatibility obligations.

Reddit-derived learning: Static-analysis and security threads repeatedly surfaced outdated packages, known vulnerabilities and the extra risk of an AI inventing or automatically installing package names.

Common problem	Small explanation / what to look for
Dependency rot	Packages remain on the version first suggested by the AI and accumulate known vulnerabilities.
Known vulnerable versions	The project ships a dependency with a published security advisory or CVE.
Unnecessary package additions	A large library is installed to solve a tiny problem already handled by the framework or a few lines of code.
Hallucinated package names	The model references a package that does not actually exist or is not the intended package.
Slopsquatting risk	An attacker publishes a package under an AI-hallucinated name hoping an agent later installs it.
Compromised trusted package	A legitimate upstream dependency is taken over or publishes a malicious release.
Unreviewed transitive dependencies	A seemingly small dependency pulls in a large executable dependency tree.
Unpinned/lockfile drift	The same source revision can resolve to different dependency versions across environments.

AI Context Drift

The model's working context is temporary. As the codebase and conversation grow, architectural assumptions can disappear and later sessions can make incompatible local decisions.

Reddit-derived learning: Subreddit discussions repeatedly described context windows becoming crowded, new sessions needing reorientation and fixes drifting because earlier design reasoning was no longer present.

Common problem	Small explanation / what to look for
Context-window architecture drift	Earlier architectural constraints fall out of context and new code violates them.
Cross-session assumption drift	A new AI session interprets the same project differently from the previous session.
Agent-to-agent disagreement	Different coding agents create incompatible patterns, naming or ownership boundaries.
Missing persistent architecture rules	Critical constraints live only in chat history instead of project documentation.
Old requirements disappear	A new implementation silently drops behaviour that was specified several sessions earlier.
Context summaries lose nuance	Compressed handoffs omit exceptions, tradeoffs or "do not change" constraints.
Local fix, global inconsistency	The model solves the file in front of it without tracing the wider execution/data flow.

Specification Drift & Design Documentation

A written specification gives humans and agents a persistent reference for intended behaviour, boundaries and acceptance criteria. Without one, the implementation becomes the only definition of the product.

Reddit-derived learning: Useful workflow threads recommended planning before coding, maintaining a living spec/PRD and building micro-feature by micro-feature rather than relying on one long conversational memory.

Common problem	Small explanation / what to look for
Requirements live only in chat	Important behaviour disappears when a session ends or context is compacted.
No acceptance criteria	There is no objective definition of “done,” so the AI chooses what success means.
Spec and implementation diverge	Code changes but the design/requirements documentation is not reconciled.
AI fills undefined behaviour	Ambiguous requirements are silently converted into implementation assumptions.
No negative constraints	The agent knows what to add but not what must not change, remove or expand.
One-shot feature planning	A large feature is implemented before the plan, data model, edge cases and tests are reviewed.
No phased/micro-feature delivery	Too much is changed at once to understand regressions or safely review the diff.
Domain knowledge not captured	Critical business rules remain in the user’s head or old prompts instead of durable project docs.

Backward Compatibility, Scope Creep & Feature Bloat

AI often tries to preserve every old interface while also adding every new idea. This can leave a codebase full of wrappers, legacy branches and features that no longer support a clear product goal.

Reddit-derived learning: A builder reported repeated backwards-compatibility creep and feature bloat until they reset around a clearer design document and explicit scope.

Common problem	Small explanation / what to look for
Unnecessary compatibility shims	Old interfaces are preserved even though no supported user or data path requires them.
Legacy branches never retired	Temporary fallback behaviour becomes permanent and keeps expanding.
Compatibility wrappers stack up	Adapters wrap adapters until the canonical path is unclear.
Scope grows faster than architecture	Fast generation encourages trying new features before existing ones are stabilized.
Prototype experiments become product code	Exploratory branches are never cleaned up after the idea is validated.
No explicit “not building” list	The project has no negative scope boundary, so every prompt can widen it.
Structural changes without easy rollback	Large refactors happen before a safe checkpoint or reviewable commit exists.

Human Review Capacity & Change Velocity

AI can change code much faster than a person can form a reliable mental model of the changes. Review quality collapses when output volume exceeds human attention.

Reddit-derived learning: A recent discussion framed the limiting context window as the human one: an agent can refactor huge areas quickly, while the developer cannot realistically verify every interaction at that speed.

Common problem	Small explanation / what to look for
Change volume exceeds review capacity	The AI edits more files and logic than a human can meaningfully inspect before the next change.
Blind diff approval	Generated changes are accepted because tests pass or the summary sounds plausible, not because the diff was understood.
Summary-based trust	The reviewer reads the agent's explanation instead of checking what the code actually does.
Too many concurrent agents	Parallel agents create interactions and merge risks faster than anyone can reason about them.
Cognitive context overload	The developer loses track of architectural state, recent decisions and which workaround depends on which.
No review checkpoints	Large tasks proceed from plan to implementation to refactor without deliberate human gates.
Review skips "unchanged" areas	The agent modifies neighbouring files or screens that were not part of the visible task and nobody checks them.

Rate Limiting, Abuse Controls & Unit Economics

A public endpoint is also a cost and abuse boundary. AI/media APIs, uploads, auth and expensive queries need explicit limits before a bot or mistake can exhaust capacity or budget.

Reddit-derived learning: Threads highlighted open endpoints, API-credit exposure and the need to understand cost per active user and worst-case request paths—not merely whether the feature works.

Common problem	Small explanation / what to look for
No API rate limiting	A loop can saturate an endpoint, exhaust resources or generate a large third-party bill.
No login/reset throttling	Authentication endpoints can be brute-forced or abused at high volume.
Unlimited AI/media calls	One user or bot can consume expensive model, image, video or processing credits.
Unlimited uploads/request size	Large or repeated payloads can exhaust storage, memory, bandwidth or moderation budgets.
No per-user quotas	The system cannot bound the cost or capacity consumed by one account.
Unknown cost per action	The team cannot estimate whether the product becomes unprofitable as usage grows.
No worst-case cost model	The design is based on average behaviour rather than malicious/retry/high-volume paths.
No abuse telemetry	There is no visibility into unusually high request, token, storage or payment activity.

Prototype vs Production Failure Paths

A prototype proves the normal path. Production must also survive invalid input, repetition, concurrency, interruption, delayed dependencies and malicious behaviour.

Reddit-derived learning: A strong recurring distinction was “demo that became a product”: refreshes mid-action, late webhooks, failed AI calls, duplicate jobs, payment cancellation and cross-user access were all examples of paths demos rarely exercise.

Common problem	Small explanation / what to look for
Normal path only	Testing proves the intended click sequence but not what happens when the sequence changes.
Invalid path missing	Bad input or impossible state is not rejected predictably.
Repeated path missing	The same action cannot be safely retried or submitted twice.
Concurrent path missing	Two users/jobs/requests mutate the same state without conflict handling.
Interrupted path missing	Refresh, network loss, process crash or cancellation leaves partial state behind.
Delayed dependency path missing	Late webhooks, queues and third-party responses arrive after local assumptions have changed.
Malicious path missing	The feature assumes a cooperative client and does not test access control or abuse.
Recovery path missing	The system detects failure but has no reconciliation, retry, rollback or operator repair process.

Production Hardening Gap

The last 20% of visible product work can hide most of the operational engineering needed for a dependable service.

Reddit-derived learning: Community discussions repeatedly separated “looks finished” from “safe to run”: production requires controls and recovery mechanisms that users rarely see in the UI.

Common problem	Small explanation / what to look for
No deployment verification	A successful build is treated as proof that the deployed service and migrations actually work.
No rollback strategy	There is no reliable way to return to a known-good application/schema version.
No alerting/ownership	Failures can occur without a responsible person/process being notified.
No capacity/load testing	Critical paths have never been exercised at realistic concurrency and data volume.
No reconciliation jobs	Payments, jobs, webhooks or external state can drift without a repair process.
No cost controls	External-service consumption is not bounded, metered or attributed.
No operational runbook	Routine incidents, restores and release failures depend on improvisation.
Development shortcuts survive launch	Debug bypasses, test data, manual patches and insecure defaults become normal production behaviour.

Comprehension Debt & Maintainability

Technical debt becomes more dangerous when nobody understands why the system was built the way it was. AI can generate working structure without transferring the reasoning behind that structure to the human owner.

Reddit-derived learning: Several posts described the hard part changing from “building” to “understanding what you built,” especially after quick fixes became dependencies for later features.

Common problem	Small explanation / what to look for
Comprehension debt	The amount of code and behaviour exceeds what any maintainer understands well enough to safely modify.
Nobody owns architectural reasoning	Important choices exist in code but no person or durable document explains the tradeoff.
Fear-driven dead code retention	Files remain because nobody can prove they are unused or safe to delete.
Quick fix becomes foundation	A temporary workaround gains downstream dependencies and becomes difficult to remove.
Maintenance requires prompt archaeology	Understanding intent depends on finding old conversations instead of reading code/tests/docs.
Onboarding cost explodes	A new maintainer must reconstruct architecture from generated code rather than a coherent design.
Root-cause tracing is slow	Incidents consume large amounts of time/tokens because the execution path is poorly understood.
Refactor risk becomes psychological	Teams avoid improving known problem areas because they cannot predict what will break.

Backup Isolation & Recovery

Version control and backups must survive the same tools and mistakes that can damage the working project. Recovery is a capability, not a folder named “backup.”

Reddit-derived learning: One “day one” discussion recommended keeping major-change backups outside the agent-accessible workspace; other threads emphasized Git and actual restore tests.

Common problem	Small explanation / what to look for
Backups inside the writable workspace	The same agent or command that damages the project can also rewrite/delete the backup.
Only local Git history	A lost/corrupted machine can remove both the project and its recovery history.
Backup job never restore-tested	Successful backup status is mistaken for proof that the data can be recovered.
Database backup not coordinated with app/schema	A restored database may not match the deployed code or migration level.
No pre-change checkpoint	Major structural work starts without a known-good commit/tag/snapshot.
Secrets included in broad backups	Recovery copies unintentionally spread credentials into less-controlled locations.
No recovery time objective	The team has no expectation for how quickly a failure must be restored.

Environment Separation & Safe Experimentation

Validated production behaviour should be isolated from experiments. AI-generated “cleanup” or feature work should not mutate live logic or data before it passes separate checks.

Reddit-derived learning: A recurring recommendation was to freeze stable behaviour, test new ideas separately and avoid using the same environment for experimentation and production.

Common problem	Small explanation / what to look for
Development and production share configuration	A change intended for testing can affect live services, secrets or data.
AI edits live logic directly	Experimental refactors are performed against the production branch/environment.
Test and live third-party credentials mixed	Payments, email, AI and storage environments can accidentally call real services.
No staging/sandbox path	There is nowhere realistic to verify migrations, integrations and deployment behaviour before launch.
Production data used casually in development	Testing can modify or expose real user information.
Validated flows repeatedly “cleaned up”	Working code is rewritten for aesthetics without a user/business reason or regression protection.
No promotion process	There is no explicit path from experiment → reviewed change → test → deploy.

Expanded Learnings from r/vibecoding Discussions

Recurring themes from the original three threads plus the wider subreddit review, synthesized into practical engineering checks. Repetition across posts is treated as a signal for what deserves explicit review—not as proof of prevalence.

Common problem	Small explanation / what to look for
Ship only what you can operate	A visually complete app is not production-ready until failures, recovery, security, cost and observability are designed.
Data-model drift is a first-class risk	Duplicated fields, null-heavy schemas and different sources for one concept make later features fragile.
Happy-path success hides most real failures	Refreshes, retries, delayed webhooks, duplicate jobs and failed dependencies expose demo-only assumptions.
Observability must precede blind re-prompting	Logs, request IDs and error tracking give AI and humans evidence instead of symptoms.
Idempotency matters for real users	Important writes, jobs and webhook handlers must tolerate repetition without duplicate side effects.
Schema migrations belong in deployment	Code and database versions must be synchronized and tested on realistic data.
Query patterns should reshape the database	Indexes, N+1 behaviour and constraints must be reviewed after real access patterns emerge.
Dependency installation is a security boundary	Audit versions, vulnerabilities, provenance and AI-installed or hallucinated packages.
Specs preserve intent across context windows	PRDs, architecture notes, acceptance criteria and negative constraints reduce AI/session drift.
AI context and human context both have limits	More generated change is not useful if nobody can review, understand or safely maintain it.
Comprehension debt is real technical debt	Code nobody understands well enough to debug is expensive even when it currently works.
Rate limiting protects both availability and money	AI/media APIs, auth, uploads and expensive queries need quotas, throttles and cost visibility.
Backups need isolation and restore proof	Keep recoverable history outside the blast radius and test restoration, not merely backup creation.
Experiment away from production	Freeze validated flows and use separate environments/credentials for risky changes and AI-led refactors.
Controlled incremental change beats broad rewrites	Plan first, change small areas, checkpoint frequently and avoid cleanup/refactors that touch working behaviour without a concrete reason.

Source note: these are synthesized community observations and include disagreement/counterpoints. They are used as experience reports and audit prompts, not controlled evidence about all AI-assisted codebases.

Priority Audit Checklist

If time is limited, inspect these areas first because they combine high impact with frequent real-world failure modes.

Common problem	Small explanation / what to look for
1. Secrets, credentials & supply chain	Search source/Git history; audit dependencies and anything an agent installs or executes.
2. Server-side authorization & tenant isolation	Verify every protected operation checks identity, role and resource ownership; actively cross-test two users.
3. Authentication/session safety	Review session expiry, reset flows, invalidation and use of mature auth frameworks.
4. Payments, idempotency & webhooks	Verify signatures/prices server-side and ensure repeated events/actions cannot double-charge or duplicate state.
5. Database model integrity	Find data-model drift, duplicate semantic fields, weak constraints, null-heavy state and ownership gaps.
6. Migrations & deployment synchronization	Test schema changes, migration ordering, rollback/forward-fix and environment version parity.
7. Backups & restore isolation	Verify off-workspace/off-machine copies and perform an actual restore into a clean environment.
8. Observability & incident diagnosis	Confirm production failures have logs, request IDs, error tracking, user/action context and alerts.
9. Rate limits, abuse & cost controls	Bound auth, AI/media, uploads and expensive routes; estimate cost per active user and worst-case action.
10. Regression tests & static analysis	Protect critical flows and run automated checks for code defects, dependency vulnerabilities and unsafe patterns.
11. Duplicate systems & sources of truth	Find parallel implementations, conflicting helpers, duplicated rules and canonical-data confusion.
12. AI/spec/context drift	Check durable architecture docs, PRD/acceptance criteria and whether recent changes violated earlier constraints.
13. Human review & comprehension debt	Identify areas changing faster than anyone can understand, review or confidently delete/refactor.
14. Async, events & repeated operations	Find missing awaits, leaked subscriptions, races, duplicate jobs and unsafe retries.
15. Scale, query performance & environment separation	Test realistic data/concurrency and confirm experiments, credentials and deployments are isolated from production.

Suggested Codex / AI Codebase Audit Prompt

Use this before asking the agent to fix anything. The aim is to discover competing systems and root causes before generating more code.

Do not fix anything yet. Audit the entire codebase as a production-readiness and maintainability review. Identify duplicate systems, competing sources of truth, dead code, architecture drift, patch-on-patch logic, data-model drift, unsafe state management, async/threading/event lifecycle problems, idempotency failures, swallowed exceptions, missing observability, security weaknesses, authorization/tenant-isolation gaps, dependency/supply-chain risks, persistence problems, schema/migration risks, scaling/query-performance problems, environment/deployment weaknesses, comprehension debt, and code that recreates existing functionality.

For each issue:

1. Identify the exact files, classes, methods, tables, migrations, endpoints or configuration involved.
2. Explain the observable symptom and the underlying root cause.
3. Rate severity as Critical, High, Medium or Low.
4. State whether a canonical implementation/source of truth already exists elsewhere.
5. Identify duplicate, legacy, compatibility or competing implementations that should be consolidated or removed.
6. Explain regression, data-loss, security and deployment risk of changing it.
7. Recommend the smallest safe fix or consolidation, but do not implement it yet.
8. Identify automated tests, observability or migration checks that should exist before the fix.
9. State how the issue behaves under six paths: normal, invalid, repeated, concurrent, interrupted and malicious.

Pay special attention to: secrets in source/Git history; UI-only security; cross-user data access; custom authentication; client-trusted payments; missing rate limits; unknown API cost per user; duplicate webhook/job processing; data-model drift; missing/over-broad indexes; N+1 queries; schema migrations not run during deployment; dependency CVEs; hallucinated or unnecessary packages; no request IDs/error tracking; untested or agent-accessible backups; production and experiment environments mixed together; silent AI rewrites; compatibility-shim accumulation; specification/context drift; full-table loading/no pagination; timezone handling; and production-only failure paths.

Also assess comprehension debt: identify areas where the code's complexity/change history exceeds the team's documented understanding, and call out any large AI-generated changes that are too broad for meaningful human review.

Finish with a prioritized remediation plan that fixes root causes before cosmetic or feature work. Do not create new abstractions, compatibility layers or parallel implementations unless you can demonstrate why the existing design cannot safely be reused.

Sources & Method

The main checklist is a software-engineering synthesis. The original three user-supplied discussions and the additional r/vibecoding threads below were reviewed for recurring failure modes, production lessons and workflow counterpoints. Individual posts are anecdotal; themes are included because they suggest useful audit checks.

R1 — What problems do you most frequently face when building and scaling vibe-coded/no-code products?

https://www.reddit.com/r/vibecoding/comments/1odi2wr/what_problems_do_you_most_frequently_face_when/

Key themes used: feature changes becoming disproportionately difficult, each change breaking something else, maintenance/evolution costing more than the first build, context limits, patch-on-patch fixes and the value of learning enough to apply AI suggestions deliberately.

R2 — I've cleaned up a dozen vibe-coded apps this year. The same 7 problems show up every single time

https://www.reddit.com/r/vibecoding/comments/1uypcye/ive_cleaned_up_a_dozen_vibecoded_apps_this_year/

Key themes used: hard-coded secrets, UI-only security, cross-user data exposure, missing error tracking, restore-untested backups, client-trusted payments, silent rewrites, weak auth patterns, pagination, timezone issues and migration testing.

R3 — Your vibe coded app is not good. There are way too many things you need to fix and you know it...

https://www.reddit.com/r/vibecoding/comments/1pyt51p/your_vibe_coded_app_is_not_good_there_are_way_too/

Key themes used: backend accountability, debugging and verification, the danger of shipping software you do not understand, security/testing expectations, plus the useful counterpoint that poor infrastructure predates AI and that low-risk internal tools can justify a different level of engineering rigor.

Additional r/vibecoding Sources

Additional threads used for the expanded audit findings and source synthesis.

R4 — if your vibe-coded app has users.. read this!

https://www.reddit.com/r/vibecoding/comments/1pn9m85/if_your_vibecoded_app_has_users_read_this/

Key themes used: data-model drift, happy-path-only flows, zero observability, cost per active user, rate limiting and separating experiments from stable production.

R5 — I reviewed 3 vibe-coded apps as a senior engineer. Here's what I found in all of them.

https://www.reddit.com/r/vibecoding/comments/1tepu7n/i_reviewed_3_vibecoded_apps_as_a_senior_engineer/

Key themes used: leaked secrets, tenant/RLS mistakes, missing rate limits, weak error handling and the need for actionable logs after exceptions are caught.

R6 — I ran static analysis on 5 of my vibe-coded projects and categorized every issue.

https://www.reddit.com/r/vibecoding/comments/1sm278z/i_ran_static_analysis_on_5_of_my_vibecoded/

Key themes used: dependency rot, known vulnerabilities, duplicated logic and the value of automated/static analysis rather than trusting generated output by inspection alone.

R7 — vibecoded a database migration, added one nullable column, and took down the entire email table

https://www.reddit.com/r/vibecoding/comments/1vr2cb2/vibecoded_a_database_migration_added_one_nullable/

Key themes used: deployment scripts that do not run migrations, code/schema version mismatch and explicit pre-deploy checks for schema-changing diffs.

R8 — I review database schemas regularly. AI-generated ones have the same problems every time.

https://www.reddit.com/r/vibecoding/comments/1ussy0l/i_review_database_schemas_regularly_aigenerated/

Key themes used: missing/overused indexes, query-pattern awareness, N+1 queries, normalization and database design that must evolve after real usage appears.

R9 — If your vibe coded app looks finished but feels impossible to safely change, read this before you rebuild everything

https://www.reddit.com/r/vibecoding/comments/1u84q8c/if_your_vibe_coded_app_looks_finished_but_feels/

Key themes used: prototype-vs-production gap, retries/late webhooks/duplicate jobs, tenant isolation, data-model coherence, freezing validated flows and sandboxing risky changes.

R10 — What's your absolute best "I wish I knew this on day one" advice for Vibecoding?

https://www.reddit.com/r/vibecoding/comments/1sw6elb/whats_your_absolute_best_i_wish_i_knew_this_on/

Key themes used: plan-before-act, Git/version control and backups kept outside the agent's writable blast radius.

R11 — Vibe coding for 30 days, 200+ hours, 70k lines as a non-developer — lessons I'd give myself on day one

https://www.reddit.com/r/vibecoding/comments/1t98ija/vibe_coding_for_30_days_200_hours_70k_lines_as_a/

Key themes used: start with a PRD, break work into micro-features, keep context focused, build tests and explicitly constrain refactors from deleting existing behaviour.

R12 — Spec Docs - Let's Discuss

https://www.reddit.com/r/vibecoding/comments/1pi87vz/spec_docs_lets_discuss/

Key themes used: living specifications, persistent domain knowledge, phased development and using durable docs to reduce cross-session context drift.

R13 — Security tip - audit your package.json for outdated/vulnerable dependencies before launching

https://www.reddit.com/r/vibecoding/comments/1qdxv4q/security_tip_audit_your_packagejson_for/

Key themes used: every dependency is executable attack surface, dependency vulnerability scanning and keeping packages patched after initial generation.

R14 — Everyone worries about AI writing insecure code. I'm more worried about what it installs.

https://www.reddit.com/r/vibecoding/comments/1v2o1xj/everyone_worries_about_ai_writing_insecure_code/

Key themes used: compromised packages, AI-installed dependencies, hallucinated package names and slopsquatting/supply-chain risk before the app even runs.

R15 — Mistakes made while developing my own app

https://www.reddit.com/r/vibecoding/comments/1sxrp6f/mistakes_made_while_developing_my_own_app/

Key themes used: feature bloat, unnecessary backwards compatibility, planning/design documents, negative scope constraints and controlled incremental changes.

R16 — Human context limits vs AI context limits

https://www.reddit.com/r/vibecoding/comments/1vwmm13/human_context_limits_vs_ai_context_limits/

Key themes used: human cognitive/review capacity can become the bottleneck even when AI can make very large changes quickly.

R17 — Vibe coding nightmare

https://www.reddit.com/r/vibecoding/comments/1vaegea/vibe_coding_nightmare/

Key themes used: technical/comprehension debt, nobody understanding a large generated codebase, difficult root-cause tracing and the value of incremental controlled development and documentation.

R18 — Vibe coding works until someone else has to maintain the vibe

https://www.reddit.com/r/vibecoding/comments/1vy04ft/vibe_coding_works_until_someone_else_has_to/

Key themes used: duplicate helpers, oversized components, fear of deleting code, quick fixes becoming foundations and maintainability after the original development “flow” is gone.

Prepared as a practical audit reference. “Common” means commonly encountered failure patterns in software and AI-assisted development, not a measured prevalence rate. Expanded r/vibecoding review edition — 29/08/26.